



A rover learning on an exoplanet

Teaching with AI (TAI)

Mirte van der Eyden¹, Andrea López Incera², Georgios Errikos Hlapanis³

¹ Institut für Fachdidaktik, Universität Innsbruck
mirte.van-der-eyden@uibk.ac.at

² Departament de didàctica de la matemàtica i les ciències experimentals, Universitat Autònoma de Barcelona
andrea.lopez.incera@uab.cat

³ 2nd Lyceum of Kos, University of the Aegean
hlapanis@2lykko.gr, hlapanis@aegean.gr



Co-funded by
the European Union



Project funded by Erasmus+ Programme of the European Union

Subject: Robotics and programming (age 16)

Activity duration: 2h 15min

Steps:

1. Introduction: what is a rover, and how could it learn? (10 min)
2. Meet the micro:bit: first MicroPython program (10 min)
3. Assignment 1 – Calibrate the rover: forward, turn, square (30 min)
4. Introduction to reinforcement learning: environment, task, ingredients (10 min)
5. Human simulation of learning, in pairs (20 min)
6. The teacher explains the learning rule with a worked example (10 min)
7. Assignment 2a – Write the pseudocode of one round of learning (10 min)
8. Assignment 2b – Read the LLM-generated code and run it on the micro:bit (15 min)
9. Assignment 3 – Final demonstration: the cutebot learns on the grid (20 min)

Learning goals:

- Practice programming with the help of an LLM (prompt engineering).
- Learn to read and understand MicroPython code written by an LLM.
- Understand the basic ingredients of reinforcement learning (state, action, reward, Q-value) and the Q-learning update rule.
- Apply the above to program and observe a rover learning a simple navigation task.

Materials needed (per group of 2-4 students): 1 micro:bit V2 + USB cable, 1 ELECF-REAKS Cutebot + battery pack, 1 laptop with access to python.microbit.org and an LLM (e.g. ChatGPT or Gemini), 1 printed 3x3 grid (template in Section A, cells at least 15 cm wide so the cutebot's movement is clearly visible).

A rover learning on an exoplanet

A robotics and reinforcement learning lesson with AI

The challenge: A rover has just landed on an exoplanet and must travel between two research stations to keep them supplied. The round trip to Earth for a radio signal takes so long that remote control is impossible: the rover has to learn, by itself, which way to go. As a team of engineers, your job today is to build that rover's brain – first teaching it to move, and then teaching it to *learn*.

Step 1. What is a rover, and how could it learn?

 10 min

Initial ideas

 Question 1: What do you think a rover is? What is it used for?

 Question 2: Have you ever seen a robotic vacuum cleaner at home? What do you think it has in common with a rover on another planet?

 **Solution / notes for the teacher:** Guide the discussion towards: both are autonomous mobile robots that must operate without a human directly steering them at every instant. A robot vacuum does not know your house’s floor plan when you buy it – it bumps into things, senses its surroundings, and gradually finds a more efficient way to cover the room. A planetary rover faces the same problem, but on a much bigger and more dangerous scale, and with a huge communication delay to Earth. The core idea of the lesson is: *if a rover cannot be told exactly what to do at every step, it needs a way to try, fail, and improve on its own.* That is what today’s lesson is about.

 **Question 3:** Very generally, what do you think any autonomous robot (rover, vacuum cleaner, self-driving car...) needs in order to move around and act on its own?

 **Solution / notes for the teacher:** Three big ingredients, which map directly onto the hardware used today: **input** (sensors, to perceive the environment), **processing** (a “brain” that decides what to do), and **output** (motors, to act on the world). Today the micro:bit will be the brain and the cutebot will be the muscles.

Step 2. Meet the micro:bit

 10 min

The **micro:bit V2** will be the brain of your rover. It is a tiny computer with a 5×5 grid of LEDs, two buttons, and several sensors. You will program it in **MicroPython**, a version of Python designed for small devices, using the online editor at python.microbit.org. You do not need to be a Python expert: today, an LLM (ChatGPT, Gemini, or similar) will be your **co-pilot**. Your job is not to memorize syntax, but to describe clearly what you want the code to do.

Try it: Connect your micro:bit to the laptop with the USB cable, open `python.microbit.org`, and ask your AI co-pilot:

“Write a simple MicroPython program for a micro:bit that shows a smiley face on the LED display.”

Paste the code into the editor, click **Send to micro:bit**, and check that it works!

Assignment 1: Calibrate your rover

 30 min

Constructing new ideas

If the micro:bit is the brain, the **ELECFREAKS Cutebot** is the muscle that will carry it across the exoplanet with its two independent wheel motors. Slot the micro:bit into the top of the cutebot and make sure the battery pack is connected and switched on.

The golden prompting rule: Always start your prompt with *“I am programming an ELECFREAKS Cutebot with a micro:bit, using MicroPython and the Cutebot library.”* This helps the AI use the correct commands. Also give the AI the file `Cutebot.py` (already loaded in your editor) so it knows which functions are available, e.g. `ct.set_motors_speed(left_speed, right_speed)`, with speeds from -100 to 100 .

Place your rover on **cell 0** of your printed grid (top-left corner, facing right/East) for every task below.

➔ Task 1: Drive in a straight line

- ☐ Ask the AI: *“Write a script to make the cutebot drive straight forward for a short time, then stop.”*
- ☐ Does it veer left or right? Ask the AI how to compensate by giving the two wheels slightly different speeds.
- ☐ Tune the driving time so the rover moves **exactly one grid cell** and stops.

 **Question 4:** Write down your final values for the speed and the driving time (in ms) it takes your rover to cross exactly one cell. You will need these numbers later!

Task 2: Turn 90 degrees

- ☐ Ask the AI: *“Write a script to make the cutebot turn right on the spot for a short time, then stop.”*
- ☐ Adjust the turning time until the rover rotates **exactly a quarter turn** (90°).

Question 5: What is your turn time in milliseconds for a perfect 90° turn?

Task 3: Walk a square

- ☐ Combine Tasks 1 and 2: ask the AI to use a loop that repeats (drive forward, turn 90°) four times.
- ☐ Try it on your printed grid: the rover should trace a perfect square of 4 cells and end up exactly where it started, facing the same direction.
- ☐ Keep fine-tuning speed and timings until it lands back on the start cell.

Question 6: Did your rover end up back on the exact starting cell, facing the same way? If not, what did you change to fix it?

 **Solution / notes for the teacher:** There is no single “correct” value: it depends on battery level, floor friction, and the individual motors, which is exactly the point of calibration. Typical starting values used in the demonstration code are speed = 30 (out of 100), MOVE_TIME $\approx$ 200 ms per cell, TURN_TIME $\approx$ 350 ms per 90° turn – but every rover is different! If students get stuck, the reference solution below can be given as a safety net (do not hand it out immediately – guide them to reach it through their own prompting first).

```
1 from microbit import *
2 from Cutebot import *
3
4 ct = CUTEBOT()
5 SPEED = 30
6 MOVE_TIME = 200    # ms to cross one grid cell
7 TURN_TIME = 350    # ms for a 90-degree turn
8
9 def forward():
10     ct.set_motors_speed(SPEED, SPEED)
11     sleep(MOVE_TIME)
12     ct.set_motors_speed(0, 0)
13
14 def turn_right():
15     ct.set_motors_speed(SPEED, -SPEED)
16     sleep(TURN_TIME)
17     ct.set_motors_speed(0, 0)
18
19 # Walk a square:
20 for i in range(4):
21     forward()
22     turn_right()
```

This same file is provided as 01_forward.py, 02_turn90.py and 03_square.py in the *Student Materials* folder, to be used only if a group is completely stuck.

Step 3. Introduction to reinforcement learning

 10 min

Initial ideas

Now that your rover can move, it is time to teach it to find its own way. From now on, the rover will live inside a simplified world: a **3×3 grid**, printed on paper, with 9 possible cells (*states*). The rover starts at cell 0 (top-left) and its task is to reach cell 8 (bottom-right, the research station) in as few moves as possible.

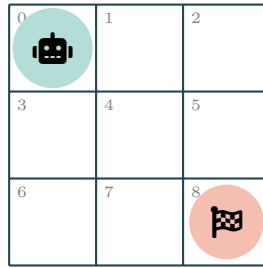


Figure 1: The 3×3 gridworld. The rover () starts at state 0, the goal () is state 8. Small grey numbers label the 9 states.

At every cell, the rover can choose one of four **actions**: move up, down, left or right (if an action would push it off the grid, it simply stays in place).

 **Question 7:** This is the kind of learning problem called *reinforcement learning* (RL): the rover is not told the correct path in advance. Based on everything we said in Step 1, what information do you think the rover needs to be able to learn the shortest path by trial and error? Think about what it must perceive, what it must remember, and what tells it whether it is doing well or badly.

 **Solution / notes for the teacher:** After students think about it and express their ideas, in a consensus, guide students towards three ingredients, which the teacher can then name explicitly:

- **Input / state:** the rover must know which cell it is currently in.
- **Memory:** the rover must remember, for every state and every possible action, some measure of “how good” that action has been so far. This is the **Q-value**: $Q(s, a)$, one number for every state-action pair. With 9 states and 4 actions, that is a table of $9 \times 4 = 36$ numbers – the rover’s entire “memory” of what it has learned.
- **Reward:** a signal that tells the rover, after each move, whether that move was good or bad, so it can update its memory.

Step 4. Human simulation: learning in pairs

 20 min

Constructing new ideas

Before the rover learns, *you* will. Work in pairs on your printed grid.

- One student is the **Rover**: close your eyes (or turn away from the grid). You do not know the grid layout, nor where the reward is. You can only call out one action at a time: “up”, “down”, “left” or “right”.
- The other student is the **Environment**: select a cell to put the reward (the Rover cannot see it). Watch the grid and, after every move the Rover calls out:
 - Tell it the cell number where it landed (state)
 - Give it feedback (reward) using this reward scheme (matching exactly what the microbit code will use later):

What happened	Reward
Normal move (moved to a new, valid cell)	−1
Tried to move off the grid (stays in place)	−5
Reached the goal cell (state 8)	+10

Play a few rounds, swapping roles. Each full attempt (from the start cell until reaching the goal) is called an **episode**. After a few rounds, the Rover can open their eyes and see where the reward was, and the Environment can tell which was the shortest path.

 **Question 8: Rover, was it easier to find the goal in your second or third attempt than in your first? Why?**

 **Solution / notes for the teacher:** Yes – this is the whole point of the activity. Even without seeing the grid, the “Rover” student starts to remember which moves led to good or bad feedback (e.g. “going right from the corner got me a -5, so I shouldn’t do that again”). That is exactly the intuition behind updating Q-values: use past experience (rewards) to make better decisions in the future.

 **Question 9: Rover, what were you memorizing to be able to learn the path to the reward?**

 **Solution / notes for the teacher:** This answer is of course open, since humans have different strategies to remember things, but the most basic thing that one needs to remember is: which action to take at which state.

 **Question 10:** Rover, did you always take the same action from the same cell, or did you sometimes try something new on purpose? Why might “trying something new” be useful, even if you already found something that works?

 **Solution / notes for the teacher:** This introduces the **explore vs. exploit** trade-off. Always repeating the first thing that worked (*exploit*) can lock the rover into a path that works but is not the shortest. Occasionally trying a random action (*explore*) lets the rover discover better paths it would otherwise never try. In the code, this is controlled by the parameter **epsilon** (ε): with probability ε the rover picks a random action; otherwise, it picks the action it currently believes is best.

 **Question 11:** How do you think the actual rover remembers where to go, based on the environment giving it a reward when it reaches the goal?

Step 5. The learning rule: Q-learning

🕒 10 min

💡 Explaining the idea (teacher-led)

The algorithm the rover uses is called **Q-learning**. The rover keeps a table $Q(s, a)$: one number for every (state, action) pair, representing how good it currently believes that action a is for that state s . The higher the Q value is for an action (at a given state), the better that action is.

The Q table is the rover's memory. At the start, the rover knows nothing, so every entry is $Q(s, a) = 0$.

After every move, the rover updates its estimate using the reward it just received. A first, simplified version of the rule would be:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r - Q(s, a)]$$

where r is the reward just received and α (the **learning rate**, between 0 and 1) controls how much the new information overwrites the old estimate. But this only looks at the *immediate* reward – it does not think ahead. The full Q-learning rule also considers how promising the *next* state looks:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

where s' is the state the rover ends up in, $\max_{a'} Q(s', a')$ is the best Q -value available from that next state (“how good does the future look from here”), and γ (**discount factor**, between 0 and 1) controls how much the rover cares about future rewards versus immediate ones.

The arrow $\leftarrow$ means that the value to its left $Q(s, a)$ changes to a new value, which is the result of the operation to its right.

A fully worked example of the rover's first decision and move. Suppose $\alpha = 0.5$, $\gamma = 0.9$, and every Q -value still equals 0 (the rover has not learned anything yet). The rover is at state 0 and, by exploring, randomly chooses the action **right**.

1. It moves to state 1 (a normal, valid move), so the reward is $r = -1$.
2. Its old estimate was $Q(0, \text{right}) = 0$.
3. The best value available from the new state ($s = 1$) is $\max_{a'} Q(1, a') = 0$ (still nothing learned there, so all Q values are zero).
4. Update the old estimate $Q(0, \text{right}) = 0$ to:
$$Q(0, \text{right}) \leftarrow 0 + 0.5 \times [-1 + 0.9 \times 0 - 0] = 0.5 \times (-1) = -0.5$$

The rover now remembers that, from state 0, going right was slightly disappointing. Over many episodes, these small updates accumulate: cells and actions that lead (even indirectly) to the goal end up with high Q -values, and the direct penalty of -5 for driving off the grid quickly teaches the rover to avoid the edges.

How the rover chooses an action: With some probability (that you choose as a programmer), the rover picks an action randomly from its options: {up, down, left, right}. Otherwise, it picks the action with the highest Q value. This is like saying, ‘I roll a die, and if it is a 6 (low probability), I pick a random action; but if it is any other value from 1 to 5 (high probability), I choose the action with highest Q value’.

 **Question 12:** Work out an example yourself, following the same steps as in the example above, to convince yourself that this works. Imagine that it is **the first time** the rover gets to the reward (cell 8). It is in cell 5 and moves down to cell 8. What would be the updated value of $Q(5, \text{down})$? On the *next* episode, how could that reward +10 “travel backwards” to help the rover in the cell next to the one just before the goal (for example, if in next episode, the rover gets to cell 2, how does it increase the Q value of going down to cell 5)?

 **Solution / notes for the teacher:** The very last action taken, the one that lands the rover on state 8, gets its Q -value pushed up directly by the +10 reward: $Q(5, \text{down}) \leftarrow 0 + 0.5 \times [10 + 0.9 \times 0 - 0] = 0.5 \times (10) = 5$. On a later episode, when the rover is in the cell *next to* the one before the goal and considers moving towards it, the $\gamma \max_{a'} Q(s', a')$ term will now be large (because that neighbouring state already has a high Q -value from the previous episode), so that action’s Q -value increases too – even though the reward for that particular move was only -1 . This is how the reward from the goal gradually “propagates backwards” across the grid over many episodes, until the whole shortest path is reflected in the Q -table.

 **Watch the Q values grow throughout the learning process.** Open the Google collab notebook ‘RL_Rover_Qvalues_Simulation.ipynb’ provided as additional material. Run all cells and choose from the points below what to do.

- Basic → Go to point 3, where you see pictures at different moments of the learning process of the highest Q value for each grid cell. You can also select which episode to see by dragging a slider.
- Advanced → Check the rest of the points, where you can see in different ways how the Q values grow, and how the rover learns.
- At the end of the notebook, there is a simulation where you watch it evolve.

Assignment 2: Program the rover's brain

 25 min (10 + 15)

Part A – Write the pseudocode  10 min

 **Question 13:** Based on everything discussed in Steps 3-5, write the pseudocode (step-by-step recipe, in your own words or simple diagrams – no need for real code yet) of *one single round* of interaction between the rover and the grid environment: from the moment it is in some state, to the moment its Q-table has been updated. Think about the steps you were doing in **one interaction** Rover-Environment (one decision + one movement) when you were the Rover and the Environment yourselves.

 **Solution / notes for the teacher:** There is more than one valid way to phrase this; the key steps to look for are:

1. **Get** the current state s (which cell the rover is in).
2. **Choose** an action a : with probability ε pick a random action (*explore*); otherwise pick the action with the highest $Q(s, a)$ (*exploit*).
3. **Perform** the action (move the rover / update its position).
4. **Observe** the new state s' and the reward r .
5. **Learn:** update $Q(s, a)$ using the Q-learning rule from Step 5.
6. **Repeat:** Set $s \leftarrow s'$ and repeat, until s is the goal state.

These steps should be given in a consensus dialogue at the end of this exercise, since they need them as such for the next exercise.

Part B – Read the LLM's code, then run it  15 min

You will be given the MicroPython code that implements exactly the pseudocode above, for the 3×3 gridworld shown on the micro:bit's top left corner 3×3 LEDs (no cutebot yet)

– just the brain, learning in simulation).

The rover will just be a light that moves on those 3×3 LEDs. The reward is on the bottom right corner of that grid, and will be shown as a dim light that is static all the time.

The prompt that created the code (roughly):

“Write MicroPython code for a BBC micro:bit that implements Q-learning on a 3×3 gridworld. States 0-8, goal at state 8. Actions: up, down, left, right. Reward -1 for a normal move, -5 for hitting the edge of the grid, +10 for reaching the goal. Show the current state as a bright pixel and the goal as a dim pixel on the 5×5 LED display. Run continuously, one training episode after another, and stop when button A is pressed.”

In the handout the code is on the left. The space on the right is for your annotations. There are already annotations in orange that will help you understand some of the things the code does. Have a look at them.

Function: A function is a piece of code that takes an input and does something with it, and often returns an output. We use functions when we want the rover to repeat a process several times, to avoid repeating all the code lines inside the function every time. A function looks like this:

```
1 def xy_to_state(x, y):  
2     return y * 3 + x
```

In this example, the input is (x,y), which are the coordinates of the grid (horizontal position, vertical position). The output is what comes after “return”, in this case $y * 3 + x$, which is the cell number. Every time we need the rover’s state to be transformed from coordinates to cell number, it will use this function.

In the code you are given, functions are highlighted in yellow and bold letters.

 On your printed handout of the code, do the following mini-challenges:

- ☐ Look for the code below the green title **Interaction Rover-Environment**. In the function `def train_episode()`, identify which line corresponds to each step of your pseudocode: Get state, Choose action, Observe new state & reward, Learn, Repeat.
- ☐ Inside the function `def train_episode()`, locate `get_next_state(state, action)`. Then find the definition of the function `get_next_state` elsewhere in the code, where it says what the function does.
- ☐ Inside the function `def get_next_state(state, action)`, locate the line where it changes the position of the rover when the rover chooses the action “UP”. It is useful to look at the coordinates that correspond to each cell, which you can find in orange, under “GRID”, in the section “Setup” at the beginning of the code.
- ☐ Inside the function `def get_next_state(state, action)`, locate where it gives a reward of +10 when it reaches the goal state.

 **If you have left-over time, feel free to try and understand more pieces of the code, and write your annotations on the right.**



Solution / notes for the teacher: Mapping of the pseudocode onto `RL_leds-3x3.py` (see also the fully annotated hand-out `Code_RL_grid_SOLUTION.docx` in the Student Materials):

- **Step 1 (Get state):** the variable `state` at the top of `train_episode()`, and `state = new_state` at the end of the loop.
- **Step 2 (Choose action):** the function `choose_action(state)` – the `if random.random() < epsilon` line is exploration, the `else` branch (`Q[state].index(max(Q[state]))`) is exploitation.
- **Step 3 (Perform action) + Step 4 (Observe s', r):** both happen together inside `get_next_state(state, action)`, which computes the new (x, y) position, checks the grid boundaries, and returns `(new_state, reward)`.
- **Step 5 (Learn):** the single line
`Q[state][action] += alpha * (reward + gamma * max(Q[new_state]) - Q[state][action])` – this is exactly the formula from Step 5!
- **Step 6 (repeat):** the `while state != goal_state:` loop.
- **Not part of the pseudocode:** `show_state()` and the LED display calls only visualise what is happening; `Q = [[0 for _ in range(4)] for _ in range(9)]` is a one-time setup (the empty “memory”), not part of a single round; the outer `while running:` loop and the button-A check just repeat episodes and let you stop the demonstration.
- The action UP inside `def get_next_state(state, action)`, is the one that subtracts 1 to the vertical coordinate (`if action == 0: new_y -= 1`).
- The lines where it gives the reward when it reaches the goal are: `if new_state == goal_state: return new_state, 10`

► **Now flash it!** Go to `python.microbit.org`, paste this code, and send it to your micro:bit. Watch the bright pixel (the rover) wander around and gradually get better at reaching the dim pixel (the goal) faster. A checkmark means an episode ended successfully. Press **button A** to stop the training loop.

🔗 **Question 14:** Watch a few episodes. Does the rover reach the goal faster over time, or does it look random the whole time? What do you think is happening inside the Q-table while you watch?

 **Solution / notes for the teacher:** Students should observe that early episodes look essentially random and often take many steps (lots of exploration and even some -5 penalties near the edges), while later episodes tend to get shorter and more direct. This is the Q-table gradually filling in with values that make the exploitation branch of `choose_action` pick better and better actions – exactly the “learning” promised in Step 3, now visible in real time.

Assignment 3: Watch your rover learn, for real

 20 min

Time to move from the simulated grid on the LEDs to the real cutebot on your printed grid. The exact same Q-learning idea now drives the wheels instead of a pixel.

- ☐ **1.** In `python.microbit.org`, make sure the file `Cutebot.py` is added to your project (Open → settings icon) – the RL code needs it to control the motors.
- ☐ **2.** Paste the RL cutebot code (below and in the file `RL_cutebot.hex`) and send it to your micro:bit.
- ☐ **3.** Place the rover on **state 0** of your printed grid, facing East (right).
- ☐ **4.** Press **button A** to start one episode (`run_episode()`).
- ☐ **5.** Watch the micro:bit’s display: it shows a ♥ (heart) every time the rover receives a positive reward, and the current state number after every move.
- ☐ **6.** If the rover drifts off the grid, revisit your calibration values (`MOVE_TIME`, `TURN_TIME`) from assignment 1.

The prompt that created this code (roughly):

“I am programming an ELECFREAKS Cutebot with a micro:bit, using MicroPython and the Cutebot library (Cutebot.py). Write Q-learning code so the cutebot physically drives around a 3x3 grid printed on paper, one cell per grid square. States 0-8, goal at state 8, actions North/East/South/West. Track the cutebot’s heading so it turns the correct way. Reward -1 for a normal move, -5 for trying to leave the grid, +10 for reaching the goal. Show a heart on the LEDs for a positive reward. Run one episode when button A is pressed.”

Listing 1: RL_cutebot.py – flashed as RL_cutebot.hex

```
1 from microbit import *
2 from Cutebot import *
3 import random
4
5 ct = CUTEBOT()
6
7 MOVE_TIME = 200          # ms for one grid cell -- CALIBRATE THIS
8 TURN_TIME = 350         # ms for a 90-degree turn -- CALIBRATE THIS
9 SPEED = 30
10
11 ALPHA = 0.5
12 GAMMA = 0.9
13 EPSILON = 0.3
14 GOAL = 8
15
16 Q = [[0,0,0,0] for _ in range(9)]
17
18 NORTH, EAST, SOUTH, WEST = 0, 1, 2, 3
19 heading = EAST
20
21 def forward():
22     ct.set_motors_speed(SPEED, SPEED)
23     sleep(MOVE_TIME)
24     ct.set_motors_speed(0, 0)
25
26 def turn_left():
27     ct.set_motors_speed(-SPEED, SPEED)
28     sleep(TURN_TIME)
29     ct.set_motors_speed(0, 0)
30
31 def turn_right():
32     ct.set_motors_speed(SPEED, -SPEED)
33     sleep(TURN_TIME)
34     ct.set_motors_speed(0, 0)
35
36 def rotate_to(target):
37     global heading
38     diff = (target - heading) % 4
39     if diff == 1: turn_right()
```

```

40     elif diff == 2: turn_right(); sleep(100); turn_right()
41     elif diff == 3: turn_left()
42     heading = target
43
44 def next_state(state, action):
45     r, c = state // 3, state % 3
46     if action == NORTH: r -= 1
47     elif action == EAST: c += 1
48     elif action == SOUTH: r += 1
49     elif action == WEST: c -= 1
50     if r < 0 or r > 2 or c < 0 or c > 2:
51         return state, -5
52     ns = r * 3 + c
53     return (ns, 10) if ns == GOAL else (ns, -1)
54
55 def choose_action(state):
56     if random.random() < EPSILON:
57         return random.randint(0, 3)
58     best = 0
59     for a in range(1, 4):
60         if Q[state][a] > Q[state][best]:
61             best = a
62     return best
63
64 def run_episode():
65     global heading
66     heading = EAST
67     state = 0
68     while state != GOAL:
69         action = choose_action(state)
70         ns, reward = next_state(state, action)
71         if reward > 0:
72             display.show(Image.HEART)
73         rotate_to(action)
74         if ns != state:
75             forward()
76         Q[state][action] += ALPHA * (reward + GAMMA * max(Q[ns])
77             - Q[state][action])
78         state = ns
79         display.show(str(state))
80         sleep(2000)
81 display.scroll("A")
82 while True:
83     if button_a.was_pressed():
84         run_episode()
85         display.show(Image.HAPPY)

```

 **Question 15:** Compare this code to the one from assignment 2. In broad terms (without going into the details of the code), what is exactly the same and what is different?

 **Solution / notes for the teacher:** Same: the Q-table structure, the reward values ($-1/-5/+10$), `choose_action` (explore/exploit with ε), and the Q-learning update line – the learning algorithm itself is identical. Different: `get_next_state` becomes `next_state` but now the “action” also has to physically happen – `rotate_to` and `forward` translate the abstract action (North/East/South/West) into real motor commands, and the code has to track the cutebot’s `heading` because, unlike a pixel that can jump directly, the rover has to physically turn before it can drive forward. This is the difference between a *simulated* agent and an *embodied* one.

 **Question 16:** Why does one single episode with the real cutebot take so much longer than one episode on the LED simulation?

 **Solution / notes for the teacher:** Physical movement (driving and turning) simply takes real time (`MOVE.TIME`, `TURN.TIME`, plus the 2-second pause built into the code to let students observe), while updating a number in the Q-table on the LED version is instantaneous. This is a very real and important limitation of embodied learning: it cannot be sped up just by having a faster processor.

 **What have we learned?**

 **Individually, write a short reflection:**

- What is reinforcement learning, in your own words, and how did your rover use it to solve the exoplanet mission?
- Where did you use AI today, and how did it help (or not help) you?

- If you had another hour, what would you try to improve about your rover's learning?

 **Question 17: Write your answers here:**

A Printable 3×3 grid template

Print one copy per group on A3 paper if possible (or tile two A4 sheets), so that each cell is comfortably larger than the cutebot ($\gtrsim 15$ cm per side). State 0 is the start (top-left), state 8 is the research station / goal (bottom-right).

0 	1	2
3	4	5
6	7	8 